

Research material part: 03

Research material part: 03

Dive deep into this research and do comprehensive refining! VPS hosted service has to be VPN gateway to network(s) connected to it. It MUST SUPPORT multiple joined networks which will be exposed through it to end users accessing these networks. So basically it establishes two ways VPN connection! We MUST have comprehensive client apps on both sides, end-user clients and for network side. Apps for configuring, customizations and administration, and access apps (like Mullvad VPN). All power features Mullvad VPN service is offering MUST BE available through this system. Especially connections obfuscation, especially Quic! Technology stack: backend and API: Go, Gin Gonic, Postgres, Redis, Podman, and everything related. Client sides, all apps - shared reusable codebases, same tech stack chosen with maximal reusability of codebase, small app sizes and small memory consumption, blazing fast and stable execution. We MUST cutting edge solution for this. Suggestions: Flutter, Go based solutions, Rust based solutions, Kotlin Multiplatform. We MUST have all apps on following platforms: Mobile (ios, android, aurora - auroraos.ru, HarmonyOS), Desktop (Windows, Linux, macOS), Web (all web browser and fully responsive). Everything MUST event driven, real time responsive! The whole system!!! Rock solid, fast, stable, game changer solution!!!

Research

Here is the comprehensive, deep-dive architectural refinement of your research into **HelixVPN**—a rock-solid, blazing-fast, event-driven, multi-network VPN gateway system.

This document transforms the raw research into a unified, cutting-edge engineering blueprint that fulfills all your requirements: multi-network exposure, two-way VPN, Mullvad-level power features (native QUIC, obfuscation), and a maximally reusable cross-platform tech stack.

HelixVPN: The Ultimate Multi-Network VPN Gateway Architecture

1. Core Architecture: The "Hub-and-Spoke" Multi-Network Model

To bypass ISP/Router inbound restrictions while exposing multiple distinct internal networks, HelixVPN uses an **Event-Driven Hub-and-Spoke** architecture.

- **The Hub (VPS Gateway):** A public VPS running the HelixVPN Backend (Go). It acts as the central traffic router, API server, and WebSocket event broker.
- **The Spokes (Network Connectors):** Agents installed on dedicated hosts inside internal networks (Home, Office, IoT networks). They establish *outbound* QUIC/WireGuard tunnels to the VPS, registering their local subnets (e.g., 192.168.1.0/24, 10.0.50.0/24).
- **The End-Users (Access Clients):** Mobile and desktop apps that connect to the VPS. The VPS dynamically routes user traffic to the appropriate internal network based on user ACLs and selected network profiles.

How Multiple Joined Networks Work

When an End-User selects "Network A" in the client app, the app sends an API request to the VPS. The VPS updates its internal routing table and pushes an AllowedIPs update via WebSocket to both the End-User client and the Network Connector, establishing a two-way routable tunnel.

2. Technology Stack: Cutting-Edge & Maximally Reusable

To achieve small app sizes, low memory consumption, and blazing-fast execution across 8+ platforms with maximum code reuse, we combine **Rust** (for the VPN core) and **Flutter** (for the UI).

Backend & API Stack (VPS)

- **Language & Framework:** Go + Gin Gonic (Blazing fast HTTP/WebSocket API).

- **Database:** PostgreSQL (Users, Networks, ACLs, Configs) + Redis (Real-time session state, Pub/Sub event bus, Rate limiting).
- **Containerization:** Podman (Daemonless, rootless, highly secure).
- **VPN Core:** Hysteria2 (QUIC-based, built-in Salamander obfuscation) as primary; WireGuard as fallback.

Client Stack (End-Users & Network Connectors)

- **VPN Core Engine:** Rust (hysteria / boringtun bindings). Rust provides memory safety, zero-cost abstractions, and compiles to native ARM/x86 binaries. This engine is shared via FFI (Foreign Function Interface) to all platforms.
 - **UI & Application Layer:** Flutter / Dart. Compiles natively to iOS, Android, Windows, macOS, Linux, and Web.
 - *Aurora OS & HarmonyOS:* Both platforms support Android runtime environments (APK compatibility) or native Flutter compilation via custom embedders, ensuring 100% code reuse from the mobile branch.
 - **Real-Time Communication:** WebSockets (Dart web_socket_channel on client, Gorilla/WebSocket on backend) for instant event propagation.
-

3. Comprehensive App Ecosystem

A. Network Connector Agent (The "Network Side" App)

A lightweight background service (CLI + minimal local Web UI) installed on internal hosts.

- **Function:** Initiates outbound QUIC connection to VPS, registers local LAN subnets, performs local NAT/forwarding.
- **Features:** Auto-reconnect, traffic shaping, local resource monitoring.
- **Platform:** Linux (Rust binary), Windows, macOS.

B. End-User Access Client (The "Mullvad-like" App)

The consumer-facing application for accessing joined networks.

- **Platforms:** iOS, Android, Aurora OS, HarmonyOS, Windows, Linux, macOS.
- **Features:**
 - 1-click network selection (Choose between exposed networks).
 - Kill-Switch (Blocks all non-VPN traffic).

- Split-Tunneling (Route only specific apps or IPs through the VPN).
- Obfuscation toggle (Enable/disable QUIC masquerading).
- Real-time latency and traffic graphs.

C. Web Admin Console (The "Configuration" App)

A fully responsive web application built with Flutter Web.

- **Function:** Manage users, rotate keys, deploy new Network Connectors via 1-click scripts, view real-time Prometheus/Grafana metrics embedded directly in the dashboard.
- **Access:** Hosted securely on the VPS behind Nginx reverse proxy with IP whitelisting and 2FA.

4. Mullvad-Level Power Features & QUIC Implementation

HelixVPN MUST match Mullvad's feature set. Here is how the system implements them:

Feature	Implementation in HelixVPN
QUIC Protocol	Hysteria2 core. Uses UDP/443, wraps traffic in HTTP/3. Brutal congestion control for high-loss networks.
Connection Obfuscation	Hysteria2 Salamander obfuscation masks VPN traffic as standard HTTPS web traffic. Masquerade proxy points to a real website to fool Deep Packet Inspection (DPI).
Kill-Switch	Client-side Rust core injects strict iptables (Linux) / WFP (Windows) / NEFilterPacketProvider (iOS) rules dropping all non-tunnel traffic if the QUIC handshake drops.
Split Tunneling	Client UI allows selecting specific apps or IPs. Rust core dynamically updates AllowedIPs and OS routing tables.
Multi-Hop	The Go backend can chain two VPS instances. Traffic flows: Client -> Entry VPS -> Exit VPS -> Internal Network.
No-Logging	Backend configured to log-level error. Redis session data lives in RAM and expires on disconnect. No traffic destination logs are stored in Postgres.

5. Event-Driven & Real-Time System Design

The entire system is event-driven, ensuring real-time responsiveness without polling.

1. **Event Bus:** Redis Pub/Sub acts as the central nervous system.
 2. **WebSocket Gateway:** Go (Gin) maintains persistent WebSocket connections to all Access Clients and Network Connectors.
 3. **Event Flow Example (Network goes offline):**
 - Network Connector loses power.
 - WebSocket connection drops on VPS.
 - Go backend detects drop -> Publishes network:offline event to Redis.
 - Go backend updates Postgres status to offline.
 - Go backend pushes WebSocket event to all Access Clients currently connected to that network.
 - Access Clients UI instantly flashes red ("Network Disconnected") and auto-fails over to another joined network if available.
-

6. Infrastructure & Deployment Blueprint

VPS Backend Deployment (Podman Compose)

Everything runs rootless via Podman for maximum security.

```
# podman-compose.yml
version: "3.8"
services:
  helix-backend:
    image: ghcr.io/yourorg/helix-backend:latest
    network_mode: host
    environment:
      - DB_HOST=localhost
      - REDIS_HOST=localhost
    cap_add:
      - NET_ADMIN
    volumes:
      - ./configs:/app/configs

  postgres:
    image: postgres:16-alpine
    environment:
      - POSTGRES_PASSWORD=supersecret
```

```

      - POSTGRES_DB=helixvpn
volumes:
  - pg_data:/var/lib/postgresql/data

redis:
  image: redis:7-alpine
  command: redis-server --save 60 1 --loglevel warning

hysteria-core:
  image: tobyxdd/hysteria:latest
  network_mode: host
  cap_add:
    - NET_ADMIN
    - NET_BIND_SERVICE
  volumes:
    - ./hysteria.yaml:/etc/hysteria/config.yaml
  command: ["server", "-c", "/etc/hysteria/config.yaml"]

volumes:
  pg_data:

```

Multi-Architecture CI/CD (GitHub Actions)

To support Mobile (ARM), Desktop (x86), and VPS (x86/ARM), the CI/CD pipeline uses Docker Buildx.

```

name: Build and Push Multi-Arch
on:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-qemu-action@v3
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${github.actor}
          password: ${secrets.GITHUB_TOKEN}
      - name: Build Backend (AMD64 + ARM64)
        uses: docker/build-push-action@v5
        with:
          platforms: linux/amd64,linux/arm64
          push: true
          tags: ghcr.io/${github.repository}/helix-backend:latest

```

7. Enterprise-Grade Resilience & Disaster Recovery

GitOps Configuration Management

All HelixVPN configurations (Network definitions, ACLs, Hysteria certs) are stored in a Git repository using **SOPS** (Secrets OperationS) backed by AWS/GCP KMS.

1. Admin updates a network ACL in the Web UI.
2. Go backend commits the change to the GitOps repo (encrypted via KMS).
3. Go backend pulls the change, updates Redis/Postgres, and live-reloads Hysteria (`kill -SIGUSR1`).
4. If a config breaks, Git revert instantly restores the last known-good state.

Automated KMS-Encrypted Backups

```
# /usr/local/bin/helix-backup.sh
set -e
BACKUP_FILE="/tmp/helix-$(date +%Y%m%d).tar.gz"
tar -czf "$BACKUP_FILE" /etc/hysteria/ /opt/helix/configs/

# Encrypt using AWS KMS (No hardcoded passwords!)
aws kms encrypt --key-id alias/helix-key \
  --plaintext fileb://"${BACKUP_FILE}" \
  --output text --query CiphertextBlob >
  "${BACKUP_FILE}.encrypted"

# Upload to S3
aws s3 cp "${BACKUP_FILE}.encrypted" s3://helix-vpn-backups/
rm -f "${BACKUP_FILE}" "${BACKUP_FILE}.encrypted"
```

Observability (Prometheus + Grafana)

- **Go Backend:** Exposes /metrics endpoint (active connections, API latency, WebSocket connections).
- **Hysteria2:** Exposes QUIC stats (bytes sent/received, packet loss, RTT).
- **Alerting:** Alertmanager pushes to Slack/PagerDuty if:
 - QUIC handshake success rate drops < 95%.
 - Network Connector WebSocket disconnects > 2 minutes.
 - VPS bandwidth exceeds 90% allocated limit.

Terraform Disaster Recovery (RTO < 15 mins)

If the primary VPS dies, a single command rebuilds the entire stack:

```
terraform apply -var="region=eu-central-1" -auto-approve
```

Terraform provisions the VPS, attaches the floating IP, and `user_data.tpl` automatically installs Podman, pulls the HelixVPN images, decrypts configs via KMS, and starts routing traffic.

Summary of the Game-Changer Aspects

1. **Zero-Touch Multi-Network Routing:** Unlike standard WireGuard which requires manual IP routing, HelixVPN abstracts this. Network Connectors broadcast subnets; the Go backend handles dynamic AllowedIPs injection to end-users.
2. **Rust + Flutter Synergy:** You get the UI fluidity of Mullvad's app with the memory safety and speed of a Rust VPN core, spanning 8+ platforms (including Aurora OS and HarmonyOS) from a single codebase.
3. **Unblockable by DPI:** By leveraging Hysteria2's QUIC + Salamander obfuscation, the VPN traffic is indistinguishable from standard HTTPS web browsing, bypassing the strictest firewalls (Great Firewall, corporate proxies).
4. **Event-Driven Precision:** Redis + WebSockets ensure that if a network drops, users know instantly and failover automatically, without relying on TCP keepalive timeouts.